
Data Mesh Foundation

A *data mesh* is a decentralized data architecture with four specific characteristics. First, it requires independent teams within designated domains to own their analytical data. Second, in a data mesh, data is treated and served as a product to help the data consumer to discover, trust, and utilize it for whatever purpose they like. Third, it relies on automated infrastructure provisioning. And fourth, it uses governance to ensure that all the independent data products are secure and follow global rules.

Although the concepts that make up a data mesh are not new, Zhamak Dehghani, CEO and founder of [Nextdata](#), deserves credit for coining the term *data mesh* and combining those concepts.¹ Although others have formed various opinions about what a data mesh is, this book bases its definition on Dehghani's work, in particular her four data mesh principles, which I discuss in this chapter. It is very important to understand that data mesh is a concept, not a technology. It is all about an organizational and cultural shift within companies. The technology used to build a data mesh could follow the modern data warehouse, data fabric, or data lakehouse architecture—or domains could even follow different architectures.

As we delve into the nuanced world of data mesh architectures in this chapter and the next, I offer a road map to navigate the complex data mesh landscape grounded in the principle of decentralization. In this chapter, I dissect the decentralized data architecture, clarifying the buzz surrounding data mesh and explicating its four foundational principles. We'll dive deep into the essence of the pure data mesh and journey through the intricacies of data domains, then discover the logical architecture and

¹ See Zhamak Dehghani, “[How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh](#),” MartinFowler.com, May 20, 2019; “[Data Mesh Principles and Logical Architecture](#),” MartinFowler.com, December 3, 2020, and *Data Mesh: Delivering Data-Driven Value at Scale* (O'Reilly, 2022).

varied topologies that support the infrastructure and draw contrasts between data mesh and data fabric. Finally, I'll discuss the proper use cases for data mesh.

Shifting focus in [Chapter 14](#), I address potential challenges in implementing a data mesh, debunking common myths to assist organizations in informed decision making regarding its adoption. Next I'll discuss conducting an organizational assessment to determine if adopting a data mesh aligns with your organization's needs. From there, I'll delve into recommendations for implementing a successful data mesh, ensuring a seamless transition and maximum benefits. Then I'll gaze ahead at the anticipated trajectory of data mesh in the ever-evolving data landscape and conclude with a discussion of when to use each of the four architectures: modern data warehouse, data fabric, data lakehouse, and data mesh.

Join me as I unravel the intricacies of data mesh architectures, providing you with a deep yet concise insight into its decentralized framework.

A Decentralized Data Architecture

The data architectures I have talked about in this book so far—modern data warehouse, data fabric, and data lakehouse—are all centralized architectures where all analytical data is created and owned by the IT team. A big difference between data mesh and those other data architectures is that data mesh is decentralized. That means the data, along with everything used to manage it, is owned and managed by individual teams or “domains” within an organization and grouped by business domain, as opposed to a central authority or single team in IT. These domain teams are responsible for collecting, processing, and managing their own data, and they have the autonomy to make decisions about how their data is used and shared. The data is kept within the domain; end users can access and query the data where it lives, without copying it to a central location. This leads to greater accountability and ensures that data is managed and maintained by those with the greatest expertise about it.

This solves the main problem with centralized architectures, which is organizational and technical scaling. As a company grows and more data has to be centralized quickly, IT often gets overloaded and becomes a bottleneck.

When I talk about all the other data architectures in this book being centralized, I'm referring to a wide variety of aspects of these architectures. Not only does a central IT team own and control data, that team is also responsible for:

- Integrating data from different sources
- Storing data in a centralized location that it owns
- Building data models and housing them in a central repository

- Governance and compliance, including defining and enforcing data quality, security, and privacy standards
- Creating and maintaining data pipelines and transformation logic
- Enforcing data quality via profiling, validation, cleansing, and other processes
- Optimizing performance
- Managing all compute technology
- Managing analytics and reporting tools
- Storing and managing metadata
- Disaster recovery and backup

In addition, in a centralized architecture, the hardware and processing power are designed to scale vertically.

Data Mesh Hype

Data mesh has drawn a lot of attention since it was first introduced in 2019. Gartner has a well-known “hype cycle” for data management, which measures the maturity of technologies based on the current level of adoption and the number of years to mainstream adoption. In its [Hype Cycle for Data Management 2023](#), it placed data mesh near the top of the hype cycle (approaching the “peak of inflated expectations”) and “moderate” on the benefit rating. Data mesh has a market penetration of 5% to 20%, and interestingly enough, Gartner predicts that “Data mesh will be obsolete before the plateau. The practice and supporting technology will evolve toward data fabric as organizations start collecting passive metadata.”

Many who hype the data mesh will claim that data warehousing projects fail at a very high rate, are unsustainable, and can no longer handle big data because they can’t scale. With all the hype, you would think building a data mesh is the answer to all of these “problems” with data warehousing. The truth is that while data warehouse projects do fail, it is rarely because they can’t scale enough to handle big data or because the architecture or the technology isn’t capable. Failure is almost always because of problems with the people and/or the process, or that the organization chose the completely wrong technology. Sometimes I feel as if data mesh is a solution looking for a problem, and the people hyping data mesh are oversimplifying the whole challenge of managing and integrating data.

I have seen big data solutions work very well for many years without data mesh, from the late 1980s (when a terabyte was considered “big data”) to today’s petabytes of data. The technology hasn’t stood still. Centralized data architectures and cloud products are quite capable of handling just about any amount of batch and streaming data.

These solutions have worked for big data for decades. Why should we assume they won't work in the future?

Of course, this is all guesswork. Data mesh is still new, but clearly not everyone believes the hype. I fall into the skeptics' camp. I have been working with databases for over 35 years and data warehousing for 15, and I've seen plenty of much-hyped technologies come and go. While I don't believe data mesh will become obsolete, I don't believe any companies will build a "pure" data mesh. A small number of companies will build a modified data mesh, and the rest will use another architecture.

I have seen very few companies put a modified data mesh into production. Many claim they have, wanting to seem cutting-edge, but most of the time it's really a data fabric or data lakehouse. But even with a very loose interpretation of "data mesh," the percentage of companies building one is very small. When I speak on data architecture, I always ask the crowd, "How many have heard of data mesh?" Usually, at least 75% raise their hands. I then follow up with, "How many are building a data mesh?" Maybe one or two hands will be raised, in a crowd of a hundred. Then I ask, "How many have a data mesh in production?" I have yet to see a single hand raised. Other colleagues report similar experiences. While this is anecdotal, it appears to indicate that data mesh is not living up to the hype. Nevertheless, data mesh architectures have a lot of positive aspects that can improve any of the other data architectures. This chapter and the next will explain in more detail.

Dehghani's Four Principles of Data Mesh

The data mesh tries to solve the four biggest challenges of centralized data architectures: lack of ownership, low data quality, technical scaling, and organizational scaling. The four principles that Dehghani lays out address these challenges.² Let's look at each in turn.

Principle #1: Domain Ownership

Principle #1 recommends that you decentralize and distribute responsibility to people who are closest to the data in order to support continuous change and scalability (for example, manufacturing, sales, supplier).

In a centralized architecture, there is usually confusion (or, in some meetings, loud arguments) about who "owns" the analytical data.

Most source data is generated by homegrown operational systems, CRM tools such as Salesforce and Microsoft Dynamics, and enterprise resource planning (ERP) tools

² See Zhamak Dehghani, *Data Mesh: Delivering Data-Driven Value at Scale* (O'Reilly, 2022), page 15 for principle #1, page 29 for principle #2, page 47 for principle #3, and page 67 for principle #4.

such as SAP and Microsoft Dynamics. Each of these domain-specific systems and applications is run by dedicated operational people who usually have little to no communication with the central IT data platform team. Often, the operational people are unaware that “their” data is being sent to a data platform.

When data is in an operational database, it belongs to the operational domain that owns the database. When it’s copied to the centralized data store (usually a data lake), IT usually owns it—but some argue that it should still belong to the domain.

For its part, the central IT data platform team rarely has a solid understanding of how the operational systems or business applications operate. They simply don’t understand the data well and have no context for how the data was generated or what it actually means. They just see table and field names or files. As a result, they struggle to produce quality data and usable analytics.

This challenge is cleared up by the first principle of the data mesh—domain ownership. The people in charge of the operational databases are *always* the owners of the analytical data, and they are responsible for managing and maintaining the data. Responsibility and ownership are decentralized and distributed to the people who are closest to the data, to support continuous change and scalability. These people are grouped according to business domain, such as manufacturing, sales, or supplier; defining a company’s domains can be challenging. Each domain owns its operational and analytical data.

Principle #2: Data as a Product

Principle #2 suggests treating analytical data provided by the domains as a product, and the consumers of that data as customers. Each domain has domain teams, API code, data and metadata, and infrastructure.

In a centralized architecture, the IT team owns the analytical data and therefore is responsible for data quality—but they usually don’t know the data well and may make mistakes transforming it. The domain team, on the other hand, has a deep understanding of the data and its context within their business domains. This knowledge allows them to identify and address data quality issues that may be missed by central IT teams, who have less context to aid in understanding the business needs.

The second principle of data mesh is that instead of thinking of analytical data as an input, an asset, or a byproduct of a process that others manage, data owners treat data as a *fully contained product* that they are responsible for. They treat the consumers of that data (data users, data analysts, data scientists) as customers. This means that domain teams apply product thinking to their data to make sure it can be easily discovered and that it’s trustworthy, accessible, secure, and understandable.

Your first step in treating data like a product is to identify exactly what a data product is in your organization. A *data product* is a self-contained, independently deployable

unit of data that delivers business value. It contains analytical data sourced from operational or transactional applications. It is designed, built, and managed by a single product team. Each data product is autonomous and managed completely independently of the other data products. It solves a particular business problem and should be designed with a clear understanding of the needs of its target users. A data product is associated with a particular data domain, so it serves a specific area of the business and should be aligned to that area's business goals. There are usually many products under each domain.

Be aware that *data product* is not the same thing as *data as a product*. *Data as a product* describes the idea that data owners treat data as a fully contained product that they are responsible for, rather than a byproduct of a process that others manage, and should make the data available to other domains and consumers. *Data product* refers to the architecture of implementing data as a product. Some within the IT industry view datasets, analytical models, and dashboard reports as data products, focusing on the physical representation of data, and do not consider any metadata, code, or infrastructure to be data products.

Note that a data product can be composed of subproducts, each responsible for providing a specific subset of data or functionality that is organized around a subject area. For example, a data product for customer data could have subproducts for transactional, demographic, and behavioral data, each managed by a separate team.

Data products are consumed through *data contracts*, which are agreements between data domains or data products that define the format and structure of the data exchange between a producer and a consumer. The data contract is a very rich document that typically contains metadata, data schemas, data transformation info, and data access rules. It also contains information on how other domains, products, or consumers will access the data product, typically through APIs, a database, a standard file format, an event stream, or a graph.

Since domain teams are directly responsible for the data, they can quickly identify and address any quality issues. This leads to faster resolution times, reducing the risk of inaccurate data being used in business decisions. Finally, domain teams are embedded within the business, so they have a better understanding of the specific data needs for their domains. This ensures that data is managed and maintained in a way that aligns with the business's needs, resulting in higher-quality and more relevant analytical data.

One more item about principle #2: each domain team is responsible for acquiring and managing its own infrastructure and resource allocations, as well as hiring people to build the architecture and manage the data. This is a big change; instead of IT cleaning and processing data for consumption (by copying operational data into analytical data), these responsibilities now fall on the domain team. The domain teams must also determine the business needs for their data. This means that each team can

determine its own resource needs based on the domain's requirements and data volume. Each team can also independently provision and scale resources as needed to handle fluctuations in demand, so multiple domain teams aren't competing for infrastructure resources from a central IT team.

Principle #3: Self-Serve Data Infrastructure as a Platform

Principle #3 recommends that you simplify data product creation and management by automating infrastructure provisioning (for example, storage, compute, data pipeline, and access control).

As I noted in the beginning of this chapter, in centralized architectures, the central IT team can become a bottleneck, making it difficult to scale effectively. This team stands up and monitors all of the architecture, including storage, compute, data pipelines, and data access policies. Often, they just don't have enough people to make data available in a timely manner.

In a data mesh, because each domain team owns its domain data and treats its data as a product, it needs its own engineering teams and infrastructure to build, deploy, monitor, and provide access to its data product. This extends domain teams' responsibility from building operational applications to include building analytics and creating a solution to share the data. In short, the first two data mesh principles impose additional effort on the domain engineering teams. This principle is about reducing that.

Domain teams might be dismayed at the thought of managing, staffing, overseeing, and paying to develop and manage their data product. To avoid frustration and push-back, you need to give them a "shortcut" to overcome some of this extra work. You don't want each domain going off and building its own infrastructure from scratch, either; "reinventing the wheel" by duplicating efforts in each domain would significantly delay putting the data mesh into production, increase costs, and risk large-scale inconsistencies and incompatibilities across domains.

To prevent these problems, a data mesh platform should be built by a dedicated, central platform team. Not everything in a data mesh is decentralized. The platform must implement all the tools and interfaces a domain-engineering team needs to simplify the lifecycle of building, testing, deploying, securing, maintaining, and sharing a data product with a consumer or with other data domains. The domain teams shouldn't need to worry about the underlying infrastructure resource provisioning. You can do this by exposing a set of standardized platform APIs or scripts that the data product developer can use to state their infrastructure needs. Then you can let the platform handle the creation and management of storage, compute, data pipelines, and access control. This will result in a standardized way of creating data products, securing them, finding them, connecting to them, and reading data from them. In a data mesh, the domain-engineering teams are made up of generalist technologists instead of

specialists, so reducing complexity through a data mesh platform can free them up to innovate with their data.

The aim of this principle is to save costs, reduce complexity, lighten the load on domain teams, reduce the need for technical specialization, and automate governance policies.

Principle #4: Federated Computational Governance

Principle #4 states that a collaborative data governance between domains and a central data team exists to define, implement, and monitor global rules (for example, interoperability, data quality, data security, regulations, and data modeling).

Data governance in a centralized architecture is much easier than in a data mesh, because with a centralized architecture approach, just one central data team owns all the data and defines, implements, and monitors all the data governance. However, this centralized approach often results in a bottleneck, slowing down the decision-making process and limiting the ability of individual domains to respond swiftly to the changing conditions and specific situations they face. It can also lead to a lack of localized expertise and contextual understanding, as the central team has to govern a vast variety of data without in-depth knowledge of each domain's nuances.

In a data mesh, a central data team defines and oversees all the data governance standards and policies, including such things as data quality, data security, regulations, and data modeling. The central data team is composed of domain representatives and subject matter experts (on compliance, legal, security, and so forth). However, implementing and monitoring data governance is left to the domains, since they own the data and know it best.

The challenge with distributing responsibilities across multiple domain teams is the potential for inconsistencies and conflicting practices, leading to operational inefficiencies and increased complexity. Domains might implement different standards for data quality (for instance, using full names of US states versus two-letter postal abbreviations). They might use different security systems or data-modeling techniques. But not all governance rules can be set globally; some are context specific and really should be set by the domain. For example, the central data team can specify that users must be authenticated, while the domain teams define what data products each user has access to.

This kind of inconsistency can lead to bad data, people seeing data they should not see, and other issues. Centralized systems often struggle with these nuances as they try to impose a one-size-fits-all policy, which can lead to misalignments and a failure to meet specific needs and demands of different departments or divisions.

That's why the concept of federated data governance, which combines centralized oversight with domain-specific autonomy, is crucial for maintaining consistency and

security across the organization. The hope is to strike a balance between domain independence and central data team oversight. Whenever possible, it helps to automate implementing and monitoring policies and to tag data for easier identification.

The “Pure” Data Mesh

Now that you understand the four principles, take a look at [Figure 13-1](#). It shows a high-level data mesh architecture that uses the four principles. The first two principles are decentralized, with each domain creating analytical data using its own IT team and infrastructure. But the other two principles require centralized teams—one to implement a data infrastructure platform and one to implement a shared governance for all the domains to use.

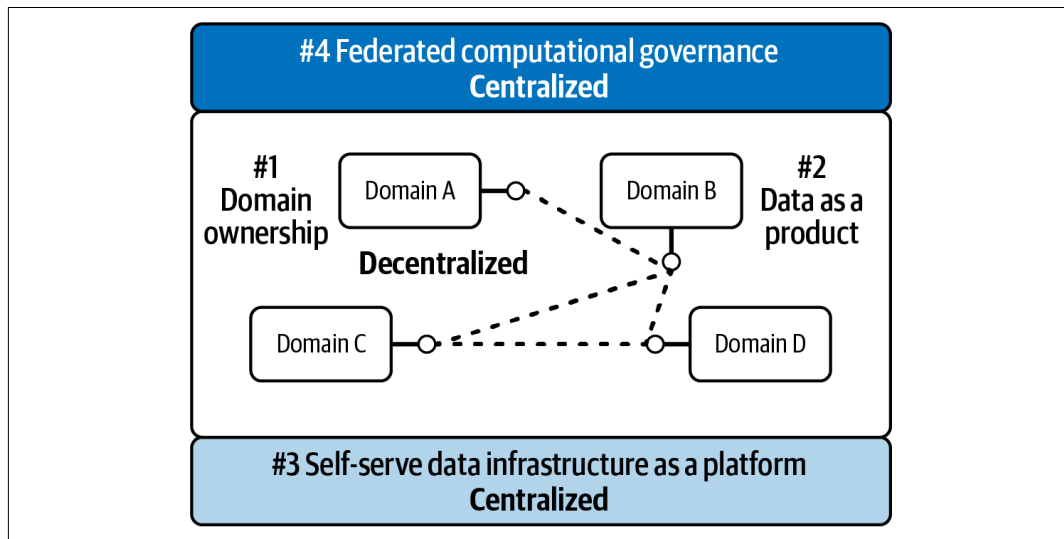


Figure 13-1. Data mesh architecture

So what makes a *true* data mesh? This is where the confusion starts. There are no universal rules on when a solution can be called a data mesh.

If a company builds a data mesh that completely follows all four principles, we would all feel confident calling that solution a “pure” data mesh. But what if the solution uses only three of the principles? Can we still call it a data mesh? What about two?

What if the solution uses all four principles, but in a way that doesn’t fulfill all the requirements of the principle? For example, you could follow principle #2, but have the domains share the same infrastructure by having one physical data lake that is logically separated into domain-specific folders that are accessible only by their respective domains, or perhaps by assigning multiple domains to one of a handful of compute clusters. As another example, using principle #3, central IT might offer scripts that help build domain storage and management, but leave the domains to

build out the rest of the infrastructure themselves. And consider one more: for principle #4, central IT could establish standards for data security and regulation, but have domains self-monitor for compliance and leave them free to do data quality and modeling as they see fit.

Some companies build a solution with two or three partially implemented principles and call it a data mesh. Others create separate data lakes for each domain and say they've built a data mesh. However, simply grouping data by business departments or organizing data into domains is not a data mesh. Even assuming they've properly designed their domains (the subject of the next section), that approach applies principle #1 but none of the others.

Some vendors will tell you that if you use their product, you will have a data mesh, even though data mesh is all about an organizational and cultural shift, not about technology. I have even heard people say that all you need to do to create a data mesh is to connect all your data lakes and data warehouses using virtualization software! Never mind that this doesn't use *any* of the four principles.

As you can see, there can be lots of “exceptions” to the pure data mesh within each principle. How many exceptions would it take for a solution not to be considered a data mesh (think of this as the minimum viable mesh)? If the four principles aren't universally accepted in total, we can't even try to answer that question. And should we all conform 100% to Dehghani's definitions or adjust them based on feedback from others in the industry? Do we need an industry committee to come up with a revised definition of a data mesh? And how do we avoid feedback based solely on self-interest?

I'm afraid I don't have all the answers. I hope this book will start a healthy discussion by providing some clarity on common data architecture concepts and the architectures that have preceded data mesh.

Data Domains

The process of designing a domain-oriented architecture, called *domain-driven design* (DDD), is very difficult and time-consuming. DDD comes from software design, but it gets much more challenging when dealing with data. The first step is to define what *domain* means in the context of your company.

According to Dehghani's first principle, analytical data should be owned by the business domains that are closest to it—either the source of the data or its main consumers. She defines three types of domain-oriented analytical data:

Source-aligned domain data

Source-aligned domain data is analytical data that corresponds closely to the operational sources where the data originates. Data is copied from the source

application's operational databases to the domain and transformed to become collaborating analytical data. This data is not fitted or modeled for a particular consumer. Source-aligned domains can have one or more sources: the data usually come from that domain's operational systems but could be operational or analytical data from other domains. For example, a company that sells books online could divide its data into domains based on the customer's purchasing journey: product search, browsing, checkout, and payment. The challenge of creating domains is ownership; for example, if a business is currently divided into application domains (domains based solely on supporting a particular application) but instead needs to be divided into business domains (based on business capabilities), a business domain could own more than one application.

Aggregated domain data

Aggregated domain data is analytical data that combines and/or aggregates data from other domains, generally to help with query performance. For example, you could combine data from the manufacturing domain and the sales domain to more easily and quickly create a profit and loss report or to create a data model that is much easier for consumers to use.

Consumer-aligned domain data

Consumer-aligned domain data is analytical data that has been transformed to fit the needs of one or more specific departments or use cases. Consumer-aligned domains almost always receive their data from source-aligned domains. For example, the domain data for a manufacturing domain can be modeled and described in two different ways: one for those who know manufacturing (source aligned) and one for those outside manufacturing (consumer aligned). Another example is transforming source-aligned data into consumer-aligned data as a way to make it easier to train machine learning models.

For more details on data domains and domain-driven design, I highly recommend [Piethen Strengolt's *Data Management at Scale*, second edition](#) (O'Reilly, 2023).

Data Mesh Logical Architecture

Figure 13-2 shows how a data mesh logical architecture could look for a company with a handful of domains. Each of these domains would have one or more products.

In **Figure 13-2**, manufacturing, sales, and supplier are all source-aligned domains. Each domain has its own operational and analytical data. For example, the sales domain could use Salesforce to track all the information about its customers (operational data). Sales could then put its data in a data lakehouse architecture (see **Chapter 12**) and combine it with other data sources that contain operational data and with analytical data from the supplier domain. This would create sales analytical data that

could then land in the data lakehouse. Sales also creates APIs to share that analytical data and its associated metadata with other domains and consumers.

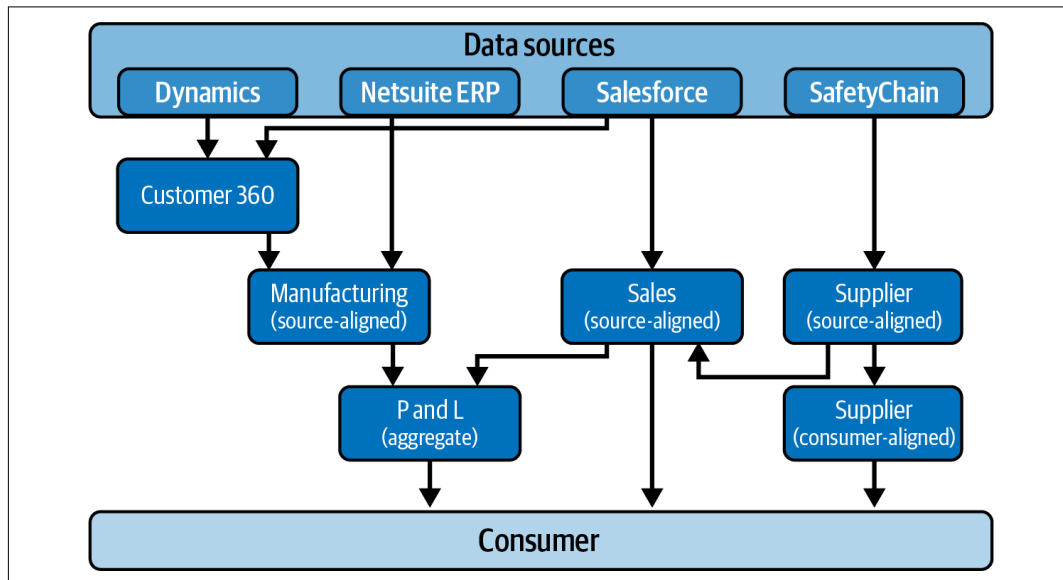


Figure 13-2. Data mesh logical architecture

Note the consumer-aligned domain for supplier. This was created because the analytical data in the supplier source-aligned domain was too complex for people outside the supplier domain to understand. In the supplier consumer-aligned domain, the data model is simplified and the supplier jargon made easier to understand.

There is also a profit and loss (P&L) aggregate domain that is built using data from the manufacturing source-aligned domain and the sales source-aligned domain. In large part, this is done for performance reasons. Executing a query or report that combines data from multiple domains can be too slow if those domains have separate data lakes that are very far apart geographically.

Finally, there is a customer 360 domain. This takes customer data (such as demographic, behavioral, transactional, and customer feedback data) from various sources, cleans and masters it (see [Chapter 6](#)), and combines it to get a full picture of the customer. All domains that need customer data pull from this customer 360 domain. This is a much better solution than having each domain pull, clean, and master customer data themselves.

As you've seen, having many domains can lead to a complicated mess. Imagine what the diagram in [Figure 13-2](#) could look like with dozens or even hundreds of domains. This is why domain design is so important. I covered it earlier in this chapter, but I encourage you to look at other resources that go into depth on this topic. You will need to spend a lot of time up front designing domains and products before you

begin building your data mesh to ensure you won't have to go back and redesign it months in.

Different Topologies

There are many possible variations or topologies of a data mesh architecture. **Figure 13-3** shows three types that I will discuss:

Mesh Type 1

In this architecture, all domains use the same technology. For example, they might all use the same cloud provider and be limited to that provider's products, so each domain would be using the same products for storage, data pipelines, security, relational data warehouses, reporting, metadata catalog, MDM, data virtualization, APIs, ML tools, and so on. There may be a few options within some products, but these choices would be minimal. Each domain would have its own infrastructure, so everything would be decentralized—except storage. Instead of each domain having its own data lake, there would be one enterprise data lake, with each domain getting its own container or folder in the lake that only it can access.

Logically, then, the domains each have their own data lakes, but physically, the data for all the domains is in one data lake. Many customers use Mesh Type 1 because of performance problems with combining data when you have multiple physical data lakes, which could be located many miles apart. In addition, having one physical data lake greatly simplifies securing and monitoring the data, as well as doing disaster recovery backups.

Mesh Type 2

In this architecture, domains use the same technology as in Mesh Type 1. Instead of one enterprise data lake for all the domain data, each domain has its own data lake. All the data lakes use the same technology. This makes the infrastructure truly decentralized, but at a cost: the technical challenge of linking all the data lakes together and getting acceptable performance when combining data from multiple domains. This is why I see more companies using Mesh Type 1.

Mesh Type 3

In this architecture, each domain can use whatever technologies and whichever cloud provider it wants and have its own data lake (which can also use any technology). You might have DomainA and DomainD using Azure, DomainB using AWS, and DomainC using GCP, while DomainA chooses Azure Synapse for its relational data warehouse and DomainD chooses SQL Server in a VM for its data warehouse. The challenges would include dealing with different types of security for each domain, finding and supporting experts for dozens of products, creating governance standards for many products in different clouds (principle #4),

creating automated infrastructure that can allow for all this variety (principle #3), *and* trying to combine data from multiple products in different clouds. This fits into the category of the pure data mesh discussed earlier. This formidable list is why I feel Mesh Type 3 will never be adopted.

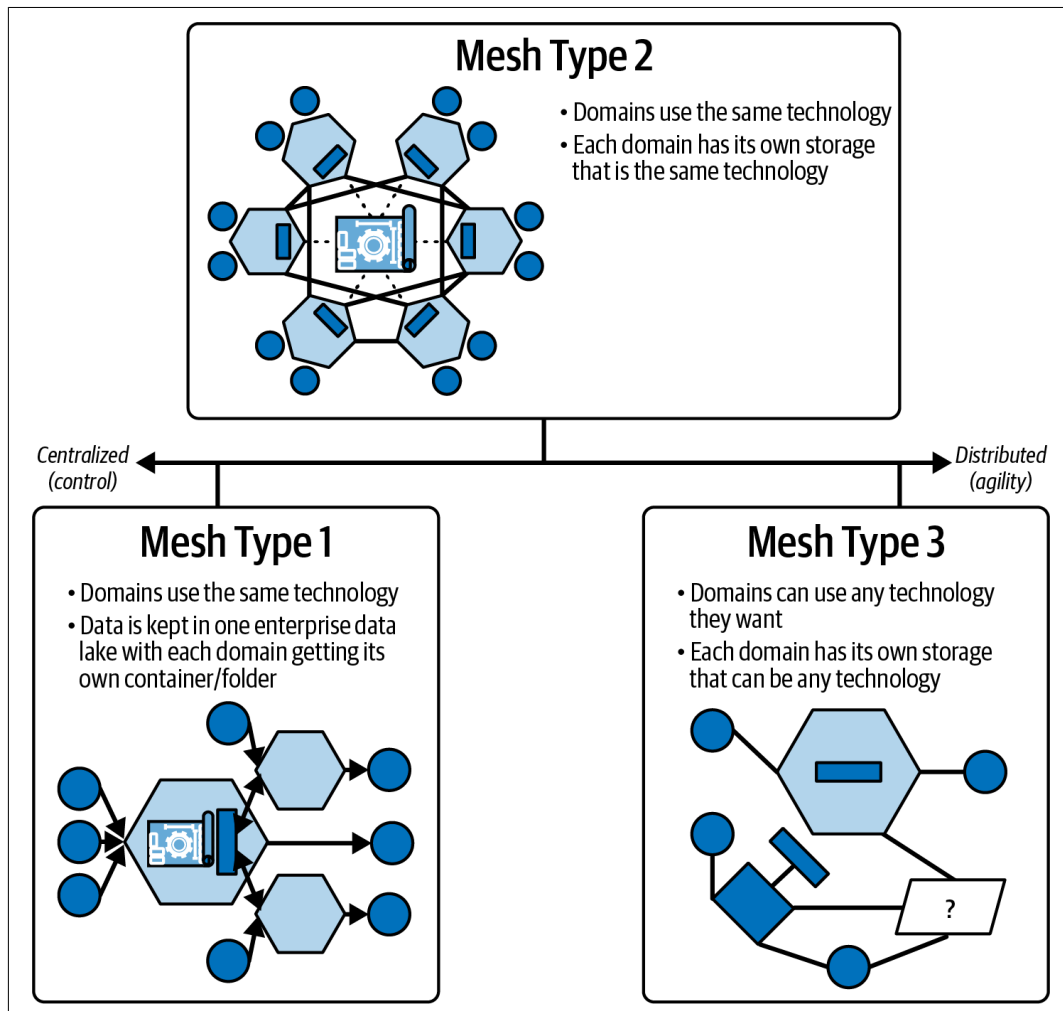


Figure 13-3. Three data mesh topologies on a spectrum from control to agility

On the left of **Figure 13-3**, the architectures have the most centralization, and the architectures become more distributed as you move to the right.

Data Mesh Versus Data Fabric

Data fabric and data mesh are both instrumental concepts in today's data landscape, but they serve distinct roles and should not be confused. At its core, a data fabric is an *architectural framework*, designed to be employed within one or more domains inside

a data mesh. The data mesh, however, is a *holistic concept*, encompassing technology, strategies, and methodologies. In fact, its definition is so encompassing that I hesitate to even label it an architecture.

On the other side of the spectrum, a data fabric's technological and architectural underpinnings can be tailored to support the various intricacies and components within a data mesh. Their relationship is not one of competition or interchangeability but rather of synergy. A single data mesh might incorporate multiple data fabrics, each tailored to the unique needs of its respective domain.

In this context, you can think of each domain in the data mesh as its own ecosystem. Within this ecosystem, the domain can build, refine, and operate its own data fabric, ensuring that it aligns with specific requirements and goals. To put it another way, while the data mesh provides a blueprint for a decentralized and domain-oriented approach to data, data fabrics furnish the architectural and technological infrastructure that powers these domains. Utilizing them in tandem can lead to a more integrated and efficient data infrastructure. This also applies to combining data mesh with a modern data warehouse or data lakehouse architecture.

Use Cases

Hopefully by now you have a good understanding of data mesh as described by Dehghani, which I will call the “pure” data mesh. While the pure data mesh sounds great in theory, it is very hypothetical and has limited technology to support it. Therefore, practical data mesh solutions will include many exceptions to Dehghani's vision. With most of the built solutions, there are so many exceptions to the pure data mesh that these “data mesh” implementations should not even be called a data mesh.

The main question to ask is this: are you trying to accomplish just data federation—integrating data from multiple, disparate sources without physically consolidating or storing the data in a single repository owned and managed by IT—or are you really trying to build a data mesh? You can accomplish data federation without building a data mesh; basic data federation can be thought of as satisfying principle #1 of data mesh and not the other three principles. If you go this route, should your solution should still be called a data mesh? I think you can call it whatever you like, but it may be more accurate to call it an enterprise data lakehouse or data fabric architecture, especially when each domain has its own workspace; the most accurate description of what you have built is that it is an enterprise data lakehouse or data fabric architecture with multiple workspaces (one workspace for each domain). Remember that you can satisfy principle #1 of a data mesh if each domain owns the analytical data derived from their operational data, even if IT creates the analytical data. However, you can't satisfy principle #2 unless each domain creates their own analytic data *and* creates the interface to share that data.

Also keep in mind that to satisfy principle #1, your organization has to have gone through the process of reorganizing your company by business domains. I question whether doing the work of creating domains is worth it if you are not also satisfying principle #2 (data as a product); if IT, not the domains, will be the cleaning that data, then you will be missing a big point of having a data mesh.

As an example, say your company acquires a lot of other companies. You have each of those acquisitions own their data, and IT use virtualization software to access it. In this scenario, you have accomplished data federation, but you certainly do not have a data mesh; you are not splitting your organization out by data domains (unless you define domains by business function and decide that acquired businesses are their own domains, which doesn't make sense because you would then, for example, have multiple HR domains with duplicate HR products). Even so, again, organizing domains based on acquisitions is only satisfying principle #1. I would argue that principle #2 needs to be satisfied to call the architecture a data mesh, as it realizes the two biggest benefits of a data mesh: organizational and technical scaling.

One last point: in the quest to strike a balance between centralization and decentralization within a data mesh, understanding the distribution of responsibilities is crucial. **Figure 13-4** is a scale that shows where each responsibility area might fit for a fictional use case.

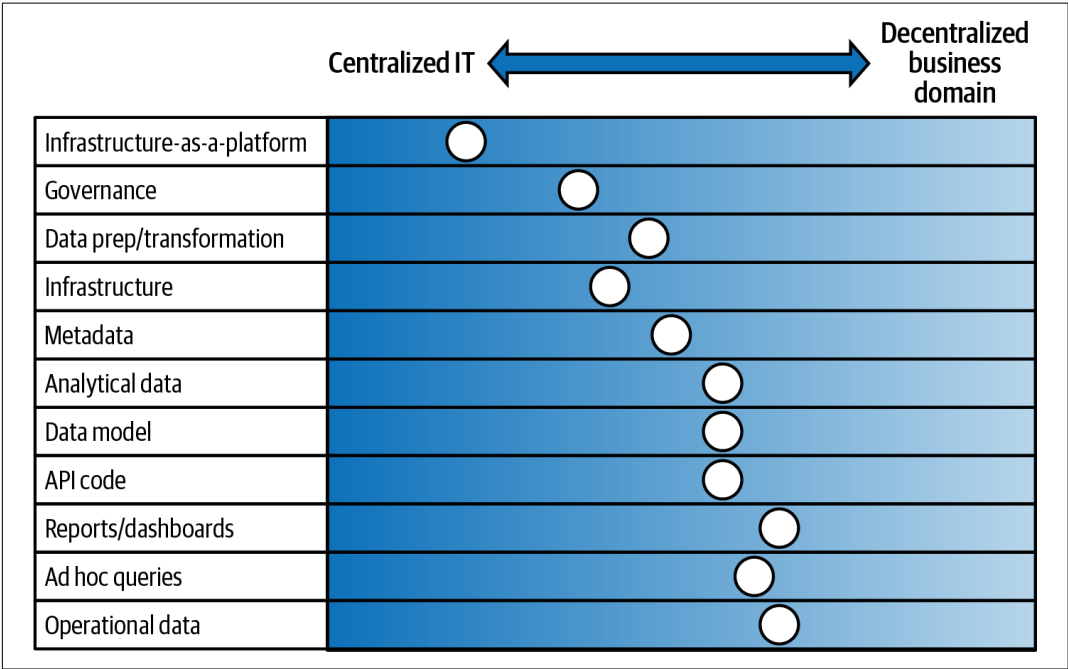


Figure 13-4. Division of responsibilities and ownership between a centralized IT department and decentralized business domains (fictional example)

On the left of [Figure 13-4](#) is centralized IT, and on the right are the decentralized business domains. A dot in the far-left means centralized IT does all of the particular item, and a dot in the far-right means each decentralized business domain does all of the particular item. Dots in between show where each responsibility area falls on this spectrum.

If most of your dots are on the far left, there is so much centralization in your solution that calling it a data mesh might not be appropriate. It might be better classified as an enterprise data fabric or data lakehouse.

Summary

This chapter provided a comprehensive overview of the data mesh concept and its fundamental principles. It began with a comparative analysis of centralization and decentralization, emphasizing the shift from a monolithic architecture to a decentralized data landscape where data is treated as a valuable product owned by cross-functional teams. I then explained Zhamak Dehghani's four principles of the data mesh: domain-oriented decentralized data ownership, data as a product, self-serve data infrastructure as a platform, and federated computational governance. These concepts highlight the importance of understanding data as a holistic entity that is owned, maintained, and utilized by specific domain teams. The idea is to treat each data unit as an individual product with its own lifecycle, governed by the team that best understands its value and use.

After detailing foundational principles, the chapter dove into the pure data mesh, emphasizing its unadulterated form. We then explored data domains, highlighting their role in structuring the decentralized environment. The data mesh logical architecture and the various topologies supporting the data mesh were discussed, showcasing their influence on data flow and interaction. A comparison between the data mesh and data fabric clarified their distinct characteristics and applications. Finally, use cases for the data mesh were discussed.

Even though the data mesh approach can be a game changer in how we handle data, it's not without hurdles and difficulties, which I'll delve into in the next chapter. I'll also address some of the common misconceptions about the data mesh and help you figure out if adopting a data mesh is the right move for your organization.

